Taller de Monitoreo y Observabilidad

Microservicios FutureX (fx-catalog-service y fx-course-service) — OpenTelemetry Collector, ELK, Prometheus, Grafana y Jaeger
Integrantes: Pablo Guzmán Alarcón, Juan David Quintero, Alejandro Troya y Stick Martinez

Este informe documenta la implementación de la observabilidad completa de los microservicios del taller: las **trazas**, las **métricas** y los **logs** se exportan con OpenTelemetry por OTLP a un único **OpenTelemetry Collector**, que los reparte a Jaeger, Prometheus y Elasticsearch (siguiendo la «Guía de Configuración de Observabilidad con OpenTelemetry Collector»). Sobre esa base se construyen los queries en ELK y Prometheus y el dashboard de Grafana que pide el taller. Entorno: Java 21, Spring Boot 3.3.3, Eureka, Resilience4j; Docker Compose con Collector contrib 0.111.0, Jaeger 1.62.0, Prometheus 2.55.1, Grafana 11.3.0 y Elasticsearch/Logstash/Kibana 8.15.3.

Evidencia	Dónde está
Pantallazos	Este documento (figuras numeradas) y carpeta `capturas/`
Videos (mp4)	Carpeta `videos/`: 00 Infraestructura y Collector · 01 Query ELK (Kibana) · 02 Queries Prometheus · 03 Dashboard Grafana, incidente y alerta
Código fuente	`monitoreo-taller-codigo-fuente.zip` (servicios, `docker-compose.yml`, configuración del Collector, Prometheus, Logstash y Grafana)
Repositorio	https://github.com/pabloandressoftware/monitoreo-taller

1. Infraestructura y exportación de telemetría (40%)

Los microservicios corren en el host; toda la infraestructura de observabilidad corre en contenedores definidos en un único `docker-compose.yml` (7 contenedores). Las aplicaciones **solo conocen al Collector**; este desacopla a los servicios de los backends:

```
Servicios (catálogo y cursos)
   │ OTLP: trazas + métricas + logs  (puertos 4319 / 4320)
   ▼
OpenTelemetry Collector
   ├─ trazas   ─► Jaeger (:16686)
   ├─ métricas ─► exporter :8889 ◄─ scrape ─ Prometheus (:9090) ─► Grafana (:3000)
   └─ logs     ─► archivo otel-collector.log ─► Logstash ─► Elasticsearch (:9200) ─► Kibana (:5601) y
Grafana
```

docker-compose.yml

```yaml
services:
  # ---------- Trazas ----------
  jaeger:
    image: jaegertracing/all-in-one:1.62.0
    container_name: jaeger
    environment:
      - COLLECTOR_OTLP_ENABLED=true
    ports:
      - "16686:16686"   # Web UI

  # ---------- OpenTelemetry Collector (trazas + métricas + logs) ----------
  otel-collector:
    image: otel/opentelemetry-collector-contrib:0.111.0   # contrib: incluye exporters prometheus y file
    container_name: otel-collector
    user: "0:0"                                           # escribe /var/log/otel-collector.log en ./logs
    command: ["--config=/etc/otel-collector-config.yaml"]
    volumes:
      - ./otel-collector-config.yaml:/etc/otel-collector-config.yaml:ro
      - ./logs:/var/log
    ports:
      - "4319:4317"   # OTLP gRPC (aplicaciones -> Colector)
      - "4320:4318"   # OTLP HTTP (métricas de Micrometer)
      - "8889:8889"   # Exporter de Prometheus
    depends_on: [jaeger]

  # ---------- Métricas ----------
  prometheus:
    image: prom/prometheus:v2.55.1
    container_name: prometheus
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
    ports:
      - "9090:9090"

  grafana:
    image: grafana/grafana:11.3.0
    container_name: grafana
    environment:
      - GF_SECURITY_ADMIN_USER=admin
      - GF_SECURITY_ADMIN_PASSWORD=admin
      - GF_AUTH_ANONYMOUS_ENABLED=true
      - GF_AUTH_ANONYMOUS_ORG_ROLE=Admin
    volumes:
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
      - ./grafana/dashboards:/var/lib/grafana/dashboards:ro
    ports:
      - "3000:3000"
    depends_on: [prometheus, elasticsearch]

  # ---------- Logs (ELK) ----------
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.15.3
    container_name: elasticsearch
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
      - ES_JAVA_OPTS=-Xms512m -Xmx512m
    ports:
      - "9200:9200"
    healthcheck:
      test: ["CMD-SHELL", "curl -s http://localhost:9200/_cluster/health | grep -qE '\"status\":\"(green|yellow)\"'"]
      interval: 5s
      timeout: 5s
      retries: 30

  logstash:
    image: docker.elastic.co/logstash/logstash:8.15.3
    container_name: logstash
    user: root
    environment:
      - LS_JAVA_OPTS=-Xms256m -Xmx256m
      - XPACK_MONITORING_ENABLED=false
    volumes:
      - ./logstash.conf:/usr/share/logstash/pipeline/logstash.conf:ro
      - ./logs:/var/log
    ports:
      - "9600:9600"
    depends_on:
      elasticsearch:
        condition: service_healthy

  kibana:
    image: docker.elastic.co/kibana/kibana:8.15.3
    container_name: kibana
    environment:
      - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
      - TELEMETRY_ENABLED=false
    ports:
      - "5601:5601"
    depends_on:
      elasticsearch:
        condition: service_healthy
```

Figura 1.1 — `docker-compose.yml`: Jaeger, Collector (puertos 4319/4320/8889), Prometheus, Grafana, Elasticsearch (con healthcheck), Logstash y Kibana

otel-collector-config.yaml

```yaml
# Guía 3: el Colector recibe OTLP (trazas, métricas, logs) y reparte cada señal a su destino
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

processors:
  batch:
    timeout: 2s

exporters:
  debug:
    verbosity: basic
  otlp/jaeger:                    # trazas -> Jaeger
    endpoint: jaeger:4317
    tls:
      insecure: true
  prometheus:                     # métricas -> Prometheus (scrape a :8889)
    endpoint: 0.0.0.0:8889
    # Sin "namespace": así la métrica se llama catalog_courses_request_seconds (como en el taller)
    resource_to_telemetry_conversion:
      enabled: true
  file:                           # logs -> archivo que lee Logstash
    path: /var/log/otel-collector.log

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [batch]
      exporters: [otlp/jaeger, debug]
    metrics:
      receivers: [otlp]
      processors: [batch]
      exporters: [prometheus, debug]
    logs:
      receivers: [otlp]
      processors: [batch]
      exporters: [file, debug]
  telemetry:
    logs:
      level: "info"
```

Figura 1.2 — `otel-collector-config.yaml`: un receptor OTLP y tres pipelines (trazas → Jaeger, métricas → Prometheus, logs → archivo)

prometheus.yml

```yaml
global:
  scrape_interval: 15s

scrape_configs:
  # Guía 3: Prometheus ya NO consulta a los servicios; solo al exporter del Colector
  - job_name: 'otel-collector'
    scrape_interval: 10s
    static_configs:
      - targets: ['otel-collector:8889']
```

Figura 1.3 — `prometheus.yml`: Prometheus solo hace scrape al exporter del Collector

logstash.conf

```
# Lee el archivo que escribe el exporter "file" del Colector (OTLP/JSON) y lo aplana para Elasticsearch.
input {
  file {
    path => "/var/log/otel-collector.log"
    start_position => "beginning"
    sincedb_path => "/dev/null"
  }
}

filter {
  json { source => "message" }

  # Una línea del Colector = resourceLogs[] > scopeLogs[] > logRecords[]: un evento por LogRecord
  split { field => "resourceLogs" }
  split { field => "[resourceLogs][scopeLogs]" }
  split { field => "[resourceLogs][scopeLogs][logRecords]" }

  ruby {
    code => '
      (event.get("[resourceLogs][resource][attributes]") || []).each do |a|
        v = a["value"] || {}
        val = v["stringValue"] || v["intValue"] || v["boolValue"]
        event.set("[service][name]", val) if a["key"] == "service.name"
        event.set("[deployment][environment]", val) if a["key"] == "deployment.environment"
      end
      rec = event.get("[resourceLogs][scopeLogs][logRecords]") || {}
      event.set("[log][level]", rec["severityText"])
      event.set("message", (rec["body"] || {})["stringValue"])
      event.set("traceId", rec["traceId"]) if rec["traceId"] && !rec["traceId"].empty?
      event.set("spanId", rec["spanId"]) if rec["spanId"] && !rec["spanId"].empty?
      (rec["attributes"] || []).each do |a|
        v = a["value"] || {}
        val = v["stringValue"] || v["intValue"] || v["boolValue"]
        event.set("[process][thread][name]", val) if a["key"] == "thread.name"
      end
      scope = event.get("[resourceLogs][scopeLogs][scope][name]")
      event.set("[log][logger]", scope) if scope
      ns = rec["timeUnixNano"].to_i
      event.set("@timestamp", LogStash::Timestamp.at(ns / 1_000_000_000.0)) if ns > 0
    '
  }

  mutate { remove_field => ["resourceLogs", "path", "host", "event", "@version"] }
}

output {
  elasticsearch {
    hosts => ["elasticsearch:9200"]
    index => "otel-logs-%{+YYYY.MM.dd}"
  }
  stdout { codec => dots }
}
```

Figura 1.4 — `logstash.conf`: lee el archivo del Collector, aplana el OTLP/JSON (service.name, log.level, message, traceId) y lo indexa en `otel-logs-*`

1.1 Instrumentación de los microservicios

En ambos servicios se agregaron `micrometer-registry-otlp` (métricas de Micrometer exportadas por OTLP) y `opentelemetry-logback-appender-1.0` (logs). La clase `OpenTelemetryConfig` construye el SDK con los **tres** exportadores OTLP gRPC (trazas, métricas y logs) hacia `localhost:4319` y conecta el appender de Logback con ese SDK; `logback-spring.xml` envía cada log como *LogRecord* con el `traceId` / `spanId` del span activo.

fx-catalog-service/.../OpenTelemetryConfig.java

```java
package com.futurex.services.FutureXCourseCatalog;

import io.opentelemetry.api.OpenTelemetry;
import io.opentelemetry.api.common.Attributes;
import io.opentelemetry.api.trace.Tracer;
import io.opentelemetry.api.trace.propagation.W3CTraceContextPropagator;
import io.opentelemetry.context.propagation.ContextPropagators;
import io.opentelemetry.exporter.otlp.logs.OtlpGrpcLogRecordExporter;
import io.opentelemetry.exporter.otlp.metrics.OtlpGrpcMetricExporter;
import io.opentelemetry.exporter.otlp.trace.OtlpGrpcSpanExporter;
import io.opentelemetry.instrumentation.logback.appender.v1_0.OpenTelemetryAppender;
import io.opentelemetry.sdk.OpenTelemetrySdk;
import io.opentelemetry.sdk.logs.SdkLoggerProvider;
import io.opentelemetry.sdk.logs.export.BatchLogRecordProcessor;
import io.opentelemetry.sdk.metrics.SdkMeterProvider;
import io.opentelemetry.sdk.metrics.export.PeriodicMetricReader;
import io.opentelemetry.sdk.resources.Resource;
import io.opentelemetry.sdk.trace.SdkTracerProvider;
import io.opentelemetry.sdk.trace.export.BatchSpanProcessor;
import java.time.Duration;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

/**
 * Guía 3: el SDK de OpenTelemetry exporta los TRES tipos de telemetría (trazas, métricas y logs)
 * por OTLP gRPC al Colector, que las reparte a Jaeger, Prometheus y Elasticsearch.
 */
@Configuration
public class OpenTelemetryConfig {

    @Bean
    public OpenTelemetry openTelemetry(@Value("${otel.service.name}") String serviceName,
                                       @Value("${otel.exporter.otlp.endpoint}") String endpoint) {
        Resource resource = Resource.getDefault()
                .merge(Resource.create(Attributes.builder()
                        .put("service.name", serviceName)
                        .put("deployment.environment", "taller")
                        .build()));

        // --- Trazas ---
        OtlpGrpcSpanExporter spanExporter = OtlpGrpcSpanExporter.builder()
                .setEndpoint(endpoint).setTimeout(Duration.ofSeconds(5)).build();
        SdkTracerProvider tracerProvider = SdkTracerProvider.builder()
                .addSpanProcessor(BatchSpanProcessor.builder(spanExporter)
                        .setScheduleDelay(Duration.ofSeconds(1)).build())
                .setResource(resource)
                .build();

        // --- Métricas (SDK de OTel; las de Micrometer salen por micrometer-registry-otlp) ---
        OtlpGrpcMetricExporter metricExporter = OtlpGrpcMetricExporter.builder()
                .setEndpoint(endpoint).build();
        SdkMeterProvider meterProvider = SdkMeterProvider.builder()
                .registerMetricReader(PeriodicMetricReader.builder(metricExporter)
                        .setInterval(Duration.ofSeconds(10)).build())
                .setResource(resource)
                .build();

        // --- Logs ---
        OtlpGrpcLogRecordExporter logExporter = OtlpGrpcLogRecordExporter.builder()
                .setEndpoint(endpoint).build();
        SdkLoggerProvider loggerProvider = SdkLoggerProvider.builder()
                .addLogRecordProcessor(BatchLogRecordProcessor.builder(logExporter)
                        .setScheduleDelay(Duration.ofSeconds(1)).build())
                .setResource(resource)
                .build();

        OpenTelemetrySdk sdk = OpenTelemetrySdk.builder()
                .setTracerProvider(tracerProvider)
                .setMeterProvider(meterProvider)
                .setLoggerProvider(loggerProvider)
                .setPropagators(ContextPropagators.create(W3CTraceContextPropagator.getInstance()))
                .build();

        // Conecta el appender de Logback (logback-spring.xml) con este SDK
        OpenTelemetryAppender.install(sdk);
        // Libera recursos al apagar la aplicación (envía lo pendiente)
        Runtime.getRuntime().addShutdownHook(new Thread(sdk::close));
        return sdk;
    }

    @Bean
    public Tracer tracer(OpenTelemetry openTelemetry) {
        return openTelemetry.getTracer("com.futurex.services.FutureXCourseCatalog", "0.0.1");
    }
}
```

Figura 1.5 — `OpenTelemetryConfig` (fx-catalog-service): exportadores de trazas, métricas y logs

fx-catalog-service/src/main/resources/logback-spring.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
    <include resource="org/springframework/boot/logging/logback/base.xml"/>

    <!-- Guía 3: cada evento de log se envía como LogRecord de OpenTelemetry (OTLP) al Colector,
         con service.name, severidad y el traceId/spanId del span activo -->
    <appender name="OTLP" class="io.opentelemetry.instrumentation.logback.appender.v1_0.OpenTelemetryAppender">
        <captureExperimentalAttributes>true</captureExperimentalAttributes>
        <captureMarkerAttribute>true</captureMarkerAttribute>
    </appender>

    <root level="INFO">
        <appender-ref ref="CONSOLE"/>
        <appender-ref ref="OTLP"/>
    </root>
</configuration>
```

Figura 1.6 — `logback-spring.xml`: appender OpenTelemetry (OTLP)

fx-course-service/src/main/resources/application.properties

```properties
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

spring.application.name=fx-course-service
server.port=8001
eureka.client.service-url.defaultZone: http://localhost:8761/eureka/

# --- OpenTelemetry (API / SDK): trazas + logs por OTLP gRPC al Colector (Guía 3) ---
otel.service.name=fx-course-service
otel.exporter.otlp.endpoint=http://localhost:4319

# --- Monitoreo: Actuator + métricas de Micrometer exportadas por OTLP al Colector ---
management.endpoints.web.exposure.include=health,metrics
management.metrics.tags.application=${spring.application.name}
management.metrics.distribution.percentiles-histogram.http.server.requests=true
management.otlp.metrics.export.enabled=true
management.otlp.metrics.export.url=http://localhost:4320/v1/metrics
management.otlp.metrics.export.step=10s
management.otlp.metrics.export.base-time-unit=seconds
management.otlp.metrics.export.resource-attributes.service.name=${spring.application.name}
management.otlp.metrics.export.resource-attributes.deployment.environment=taller
```

Figura 1.7 — `application.properties` de fx-course-service: Actuator, exportación OTLP de métricas y endpoint del Collector

Además, como pide la guía, se instrumentó el código de negocio: en `fx-catalog-service` un `Timer` por endpoint (`catalog.home.request`, `catalog.courses.request`, `catalog.firstcourse.request`, con la etiqueta `endpoint` y histograma de percentiles) y logs `INFO`/`ERROR`; en `fx-course-service` un `Counter` `courses.accessed`. Las métricas de Spring (`http.server.requests`), de la JVM (CPU y memoria) y de Resilience4j (circuit breaker) también viajan por el mismo camino.

fx-catalog-service/.../CatalogController.java (extracto)

```java
    /** Timer (Guía 3) con la etiqueta endpoint; publica histograma para calcular percentiles. */
    private Timer timer(String name, String endpoint) {
        return Timer.builder(name).tag("endpoint", endpoint)
                .description("Tiempo de respuesta del endpoint " + endpoint)
                .publishPercentileHistogram().register(meterRegistry);
    }

    // ...
    @GetMapping("/catalog")
    @Retry(name = "courseService", fallbackMethod = "fallbackGetCatalog")
    @CircuitBreaker(name = "courseService")
    public String getCatalog() {
        log.info("Received request for course catalog");
        Timer.Sample sample = Timer.start(meterRegistry);
        try {
            String courses = tracedGet("CatalogController.getCatalog", courseServiceUrl() + "courses", String.class);
            log.info("Returning course catalog");
            return "Our courses are " + courses;
        } finally {
            sample.stop(timer("catalog.courses.request", "/catalog"));
        }
    }
```

Figura 1.8 — `CatalogController`: Timer de Micrometer con etiqueta `endpoint` y logs del endpoint `/catalog`

1.2 Verificación de la exportación de los tres tipos de datos

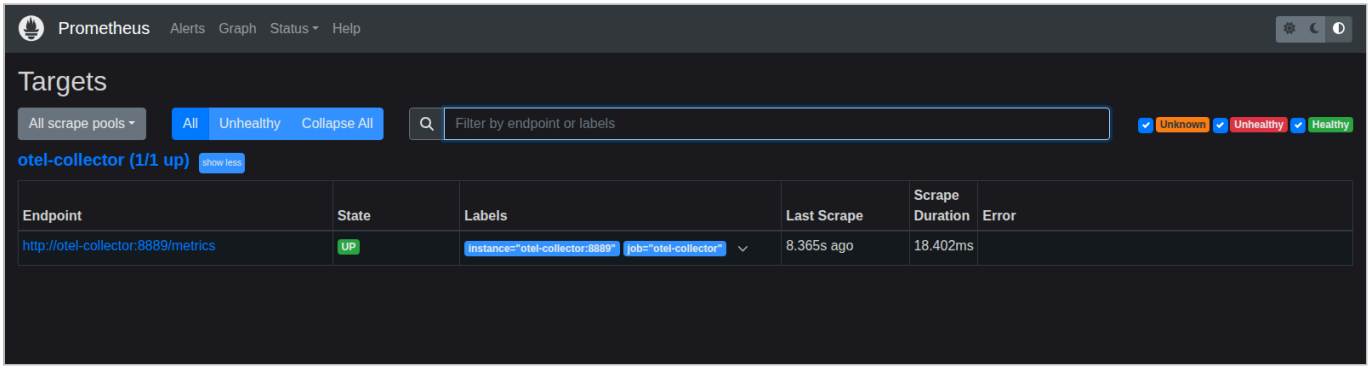


Figura 1.9 — Métricas: Prometheus (Status > Targets) obtiene los datos del exporter del Collector (`otel-collector:8889`) — estado UP

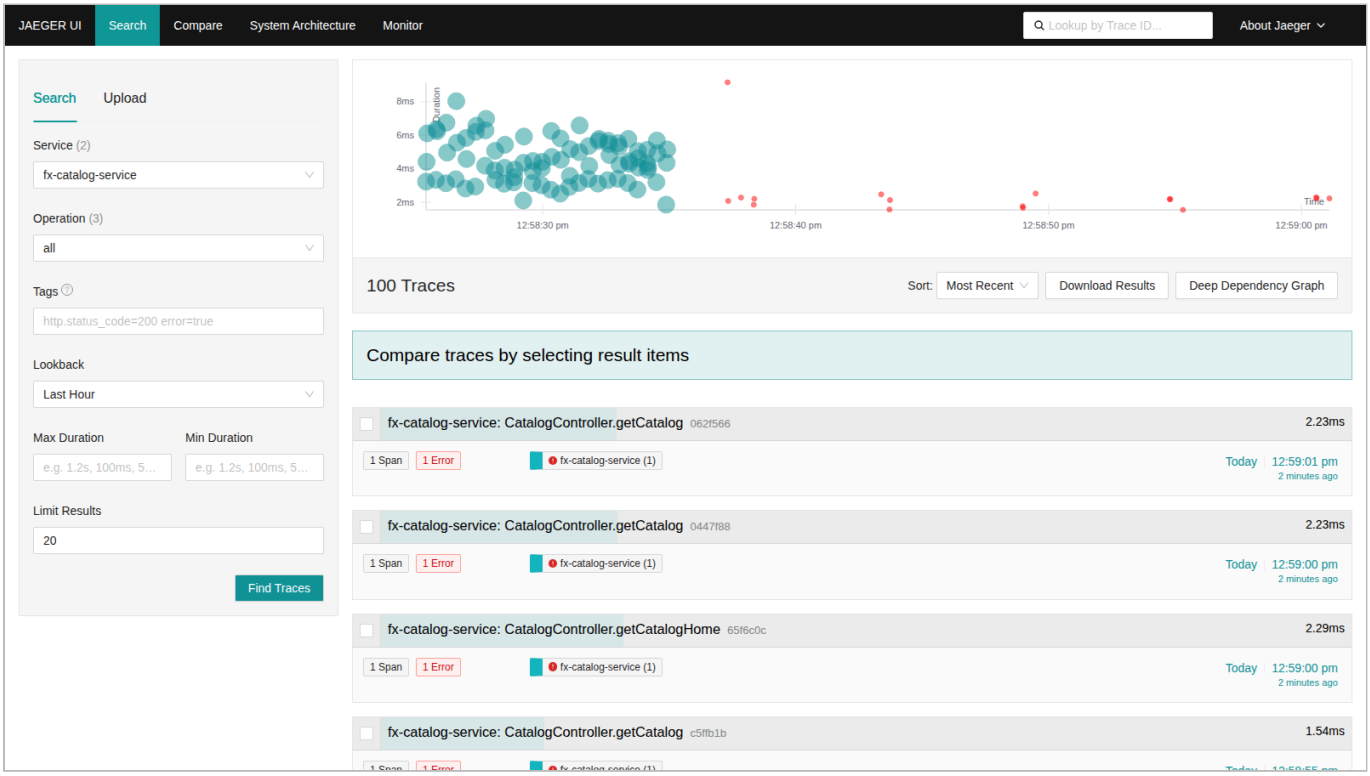


Figura 1.10 — Trazas: Jaeger muestra las trazas distribuidas catálogo → cursos que llegaron a través del Collector

Logs: ver los documentos de Elasticsearch en Kibana (Figura 2.1). El video `00_Infraestructura_Docker_Collector.mp4` muestra los contenedores en ejecución, las métricas en el exporter del Collector, el archivo de logs, el índice `otel-logs-*` y el estado de los targets.

2. Query en ELK (Elasticsearch / Kibana)

Objetivo: encontrar todos los logs de error de `fx-catalog-service` en las últimas 24 horas. Se usó el query del enunciado **sin modificarlo** (funciona porque Logstash deja los campos como `service.name` y `log.level`):

```
{
  "query": { "bool": { "must": [
    { "match": { "service.name": "fx-catalog-service" } },
    { "match": { "log.level": "ERROR" } },
    { "range": { "@timestamp": { "gte": "now-24h" } } }
  ] } }
}
```

Resultado: devuelve los eventos `ERROR` generados al detener fx-course-service y llamar a `/catalog`: cada documento trae el mensaje con la URL que falló y el `traceId`. En Kibana se creó el data view `otel-logs-*` y se usó el equivalente KQL `service.name:"fx-catalog-service" and log.level:"ERROR"` en Discover (Figura 2.1). El video `01_Query_ELK_Kibana.mp4` muestra el query en Kibana, el mismo JSON contra la API de Elasticsearch y cómo el `traceId` del log abre la traza en Jaeger.

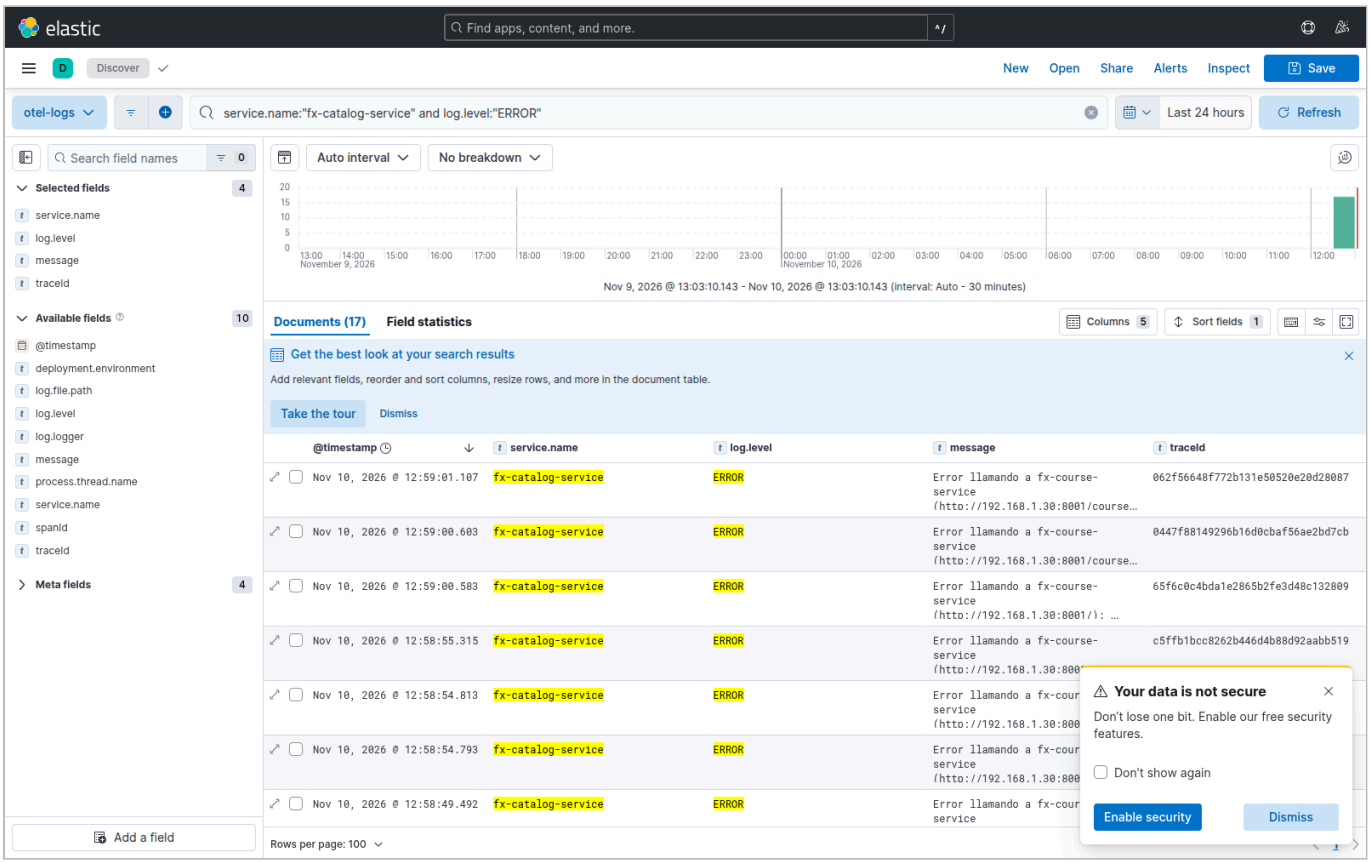


Figura 2.1 — Kibana Discover: query de errores de fx-catalog-service en las últimas 24 h

Relevancia: permite diagnosticar rápido *qué* falló y *cuándo*; con el `traceId` se salta a la traza exacta en Jaeger para ver en qué servicio se originó el problema.

3. Query en Prometheus

Objetivo: calcular el tiempo promedio de respuesta del endpoint `/catalog` en los últimos 15 minutos. La métrica `catalog_courses_request_seconds` es el `Timer` `catalog.courses.request` de la guía (con etiqueta `endpoint`), por lo que el query del enunciado funciona **tal cual**:

```
rate(catalog_courses_request_seconds_sum{endpoint="/catalog"}[15m])
/
rate(catalog_courses_request_seconds_count{endpoint="/catalog"}[15m])
```

Resultado: la gráfica muestra la latencia media de `/catalog`: ≈ 10 ms en operación normal, con un aumento cuando se detiene el servicio de cursos (reintentos de Resilience4j con espera de 500 ms). Query adicional: percentil 95, `histogram_quantile(0.95, sum by (le, endpoint) (rate(catalog_courses_request_seconds_bucket[5m])))` (ver video `02_Query_Prometheus.mp4`).

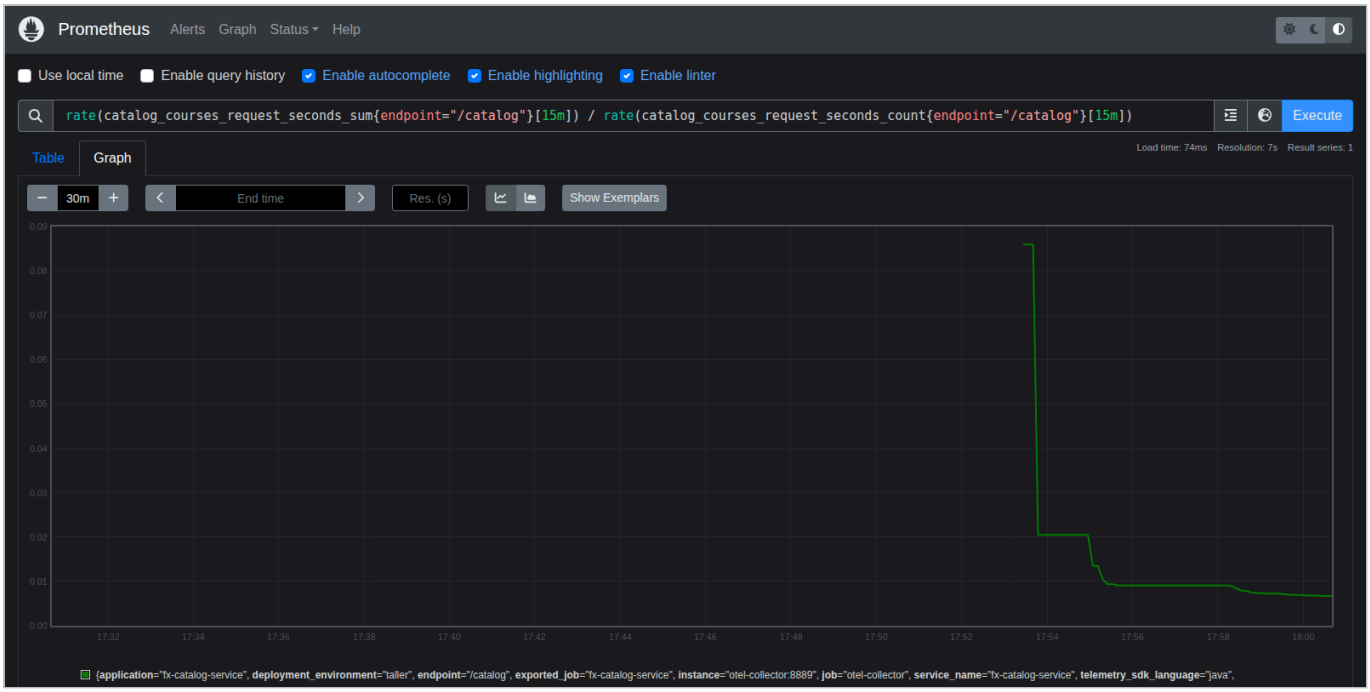


Figura 3.1 — Prometheus: tiempo promedio de respuesta de `/catalog` (ventana de 15 min)

Relevancia: muestra el impacto de una dependencia caída sobre la latencia que percibe el cliente; es la base para definir un SLO de latencia.

4. Dashboard en Grafana (provisionado como código)

El dashboard **«FX Microservicios – Observabilidad»** y los datasources (Prometheus y Elasticsearch) se cargan automáticamente desde `grafana/provisioning` y `grafana/dashboards/fx.json`; el entorno es reproducible con un solo `docker compose up`. Cubre los tres ejemplos del enunciado y añade paneles extra:

Panel	Query
Solicitudes por minuto por endpoint (Ej. 1)	`sum by (uri) (rate(http_server_requests_seconds_count{application="fx-catalog-service",uri!~"/actuator.*"}[1m])) * 60`
Latencia p95 por endpoint	`histogram_quantile(0.95, sum by (le, uri) (rate(http_server_requests_seconds_bucket{...}[1m])))`
CPU y memoria por servicio (Ej. 2)	`process_cpu_usage{application=~"fx-.*"} * 100` · `sum by (application) (jvm_memory_used_bytes{area="heap"})`
Errores vs exitosas (Ej. 3)	Resilience4j: `resilience4j_circuitbreaker_calls_seconds_count{kind="successful"}` vs `kind="failed"` + `..._not_permitted_calls_total`
% de llamadas con error · Estado del Circuit Breaker	Cociente de las series anteriores · `resilience4j_circuitbreaker_state{name="courseService"} == 1`
Tiempo promedio por endpoint (métrica de la guía)	`sum by (endpoint) (rate(catalog_courses_request_seconds_sum[1m])) / sum by (endpoint) (rate(catalog_courses_request_seconds_count[1m]))`
Cursos consultados por minuto	`sum(rate(courses_accessed_total[1m])) * 60`
Logs ERROR/WARN (Elasticsearch)	`log.level:(ERROR OR WARN)` sobre `otel-logs-*`



Figura 4.1 — Dashboard durante un incidente (fx-course-service caído): errores, circuit breaker en *open* y logs de fallback en el mismo tablero

Diseño y relevancia: la fila superior responde «¿cuánto tráfico y qué tan rápido?»; la segunda «¿cuántos recursos consumen los servicios?»; la tercera «¿está sano el servicio del que dependo?»; la cuarta usa las métricas de negocio de la guía; y el panel de logs de Elasticsearch permite correlacionar cada pico de errores con el mensaje concreto. El video `03_Dashboard_Grafana_Alerta.mp4` muestra el dashboard en vivo durante un incidente completo.

4.1 Alerta en Grafana (característica avanzada)

Se provisionó (`grafana/provisioning/alerting/alerts.json`) una regla que evalúa cada 10 s el porcentaje de llamadas fallidas hacia fx-course-service y pasa a **Firing** si supera el 5 % durante 30 s. Al detener fx-course-service y generar tráfico, la alerta pasó a Firing (Figura 4.2).

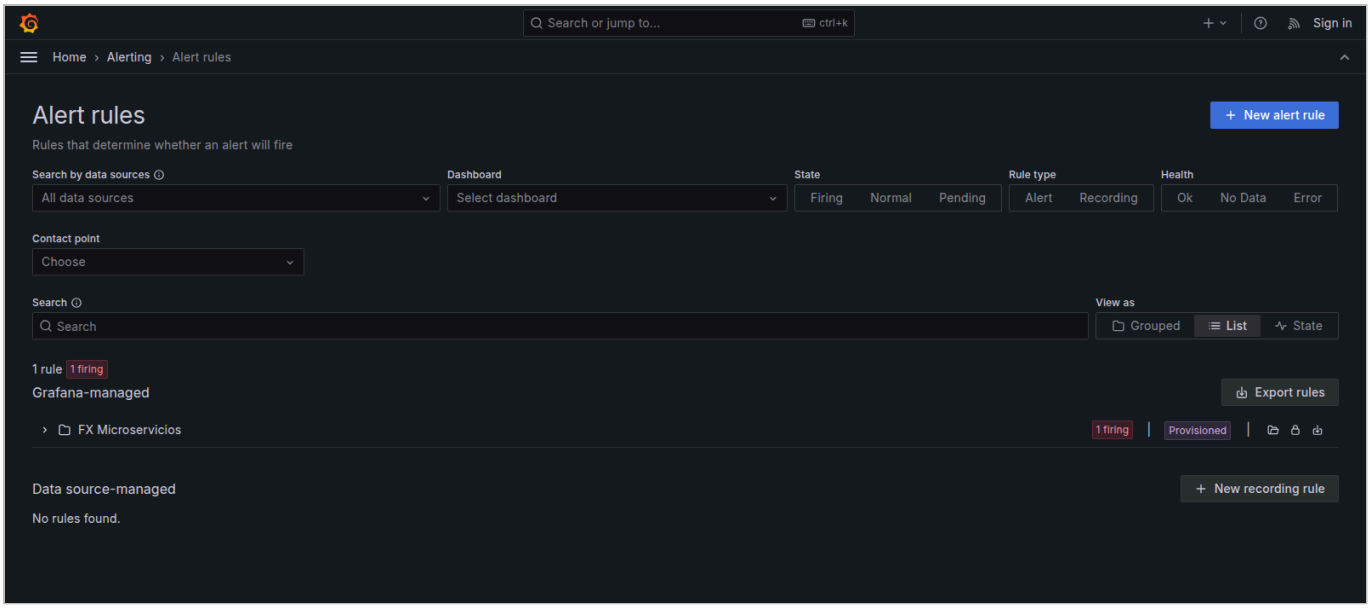


Figura 4.2 — Grafana Alerting: la regla «Tasa de errores alta hacia fx-course-service» en estado Firing

5. Escenario de correlación y análisis

Se generó carga mixta con `scripts/trafico.sh` y se simuló una falla deteniendo fx-course-service. Con los tres tipos de datos se reconstruye el incidente: (1) **Grafana/Prometheus** muestran el aumento de latencia y de errores y la apertura del circuit breaker; (2) **Kibana** muestra los logs `ERROR` / `WARN` con la causa (conexión rechazada, luego «CircuitBreaker is OPEN»); (3) **Jaeger** muestra, mediante el `traceId` del log, la traza que falló en el span cliente hacia fx-course-service. Al reiniciar el servicio el circuito pasa a half-open y luego closed, y la latencia vuelve a la normalidad.

6. Desafíos encontrados y solución

Desafío: Los errores del servicio no aparecían como HTTP 5xx
Problema: Con fx-course-service caído el catálogo responde **HTTP 200** con el mensaje del fallback (patrón Circuit Breaker); un panel con `status=~"5.."` nunca registraba errores y la alerta jamás habría disparado.
Solución: Se usaron las métricas de Resilience4j (`calls_seconds_count{kind="failed"}` y `not_permitted_calls_total`), que sí reflejan los fallos reales de la dependencia, en el panel y en la alerta.

Desafío: La guía usa imágenes que no traen todos los exporters
Problema: La guía usa `otel/opentelemetry-collector:0.88.0` (versión «core»), que no incluye los exporters `prometheus` ni `file` que la propia configuración requiere.
Solución: Se usó la imagen `opentelemetry-collector-contrib:0.111.0` . También se usó el exporter `debug` (el `logging` está obsoleto) y versiones actuales de ELK, Prometheus y Grafana.

Desafío: Nombre de la métrica en Prometheus
Problema: El Collector de la guía configura `namespace: "otel"` en el exporter de Prometheus, lo que renombraría la métrica a `otel_catalog_courses_request_seconds` y rompería el query del taller. Además el Timer de la guía no tenía la etiqueta `endpoint` que usa el query.
Solución: Se quitó el namespace, se configuró `base-time-unit: seconds` en el exporter OTLP de Micrometer (para obtener el sufijo `_seconds`) y se agregó la etiqueta `endpoint` a los Timers.

Desafío: Los logs del Collector no tenían los campos del query
Problema: El exporter `file` escribe el formato OTLP/JSON anidado (`resourceLogs → scopeLogs → logRecords`); en Elasticsearch no existían `service.name` ni `log.level` , y el query del taller no devolvía resultados.
Solución: Se escribió un filtro en Logstash (`split` ×3 + `ruby`) que genera un documento por log y mapea `service.name` , `log.level` (severityText), `message` , `traceId` , el logger (del *scope*) y `@timestamp` .

Desafío: Los logs de error no llevaban traceId
Problema: El `log.error` se ejecutaba en el `catch` , después de que el `try-with-resources` cerrara el scope del span, por lo que el log salía sin contexto de traza.
Solución: Se reactivó el span (`span.makeCurrent()`) alrededor del `log.error` para que el appender de OpenTelemetry adjunte `traceId` y `spanId` .

> **Desafío: Dependencias de la guía en conflicto**
> *Problema:* El `pom.xml` de la guía incluye `opentelemetry-spring-boot-starter` (que autoconfigura un segundo SDK y obliga a activar `allow-bean-definition-overriding`) y un appender de Logback alfa antiguo (1.31.0).
> *Solución:* Se omitió el starter (el SDK se configura a mano en `OpenTelemetryConfig`) y se usó el appender 2.9.0-alpha, alineado con el SDK 1.43.0.

> **Desafío: Logstash/Kibana no conectaban con Elasticsearch**
> *Problema:* Arrancaban antes de que Elasticsearch aceptara conexiones («Could not connect to a compatible version of Elasticsearch»).
> *Solución:* Healthcheck en Elasticsearch y `depends_on: condition: service_healthy` en Logstash y Kibana; se desactivó el monitoreo interno de Logstash. Al reingerir logs se borró el índice por nombre exacto para evitar duplicados.

7. Conclusión

Las métricas indican *qué* está pasando (latencia, errores, recursos), los logs explican *por qué* y las trazas muestran *dónde*. Centralizar la telemetría en el OpenTelemetry Collector desacopla a los microservicios de los backends (cambiar Jaeger, Prometheus o Elasticsearch no requiere tocar el código), y unir los tres tipos de datos con el `traceId` y en un dashboard de Grafana reduce el tiempo de diagnóstico. Toda la infraestructura está definida como código (Compose, Collector, Logstash, provisioning de Grafana), por lo que la solución es reproducible.

Anexo: cómo reproducir

```
cd monitoreo-taller && docker compose up -d
(cd fx-discovery-server && sh ./mvnw -DskipTests package) ; idem fx-course-service y fx-catalog-service
java -jar fx-discovery-server/target/*.jar ; java -jar fx-course-service/target/*.jar ; java -jar fx-catalog-service/target/*.jar
./scripts/trafico.sh 60
Kibana http://localhost:5601 (data view otel-logs-*) | Prometheus :9090 | Grafana :3000 (admin/admin) | Jaeger :16686
```